

STAIRCASE

A Structural Decomposition of SHA-256 Round Visibility

Original Research Contribution to the Cryptographic Community

Neerugattu Venkatesh

Freelancer / Independent Cryptography Researcher

25 July 2026

Abstract

This document presents **STAIRCASE**, an original structural framing of SHA-256 round visibility introduced by **Neerugattu Venkatesh**. It shows, in a visually clear and mathematically exact way, which internal SHA-256 state words are fixed by public constants, which are fixed by the final hash, and which remain unknown.

The document includes the front staircase for rounds 0–3, the backward staircase for rounds 57–63, percentage visibility charts, a complete worked example using `SHA256("hello")`, and a cross-platform Python verification script.

The scope is limited to standard SHA-256, single-block hashing with known IV, and structural inversion. No full-round preimage attack is claimed.

Contents

1	Plain-Language Summary	2
2	Scope and Correctness Statement	2
3	Known Prior Work and Original Contribution	2
4	SHA-256 Round Function	3
5	Constants Fixed for Every Standard SHA-256 First Block	4
5.1	Initial hash values	4
5.2	Round constants	4
5.3	A first-round constant: $T_{2,0}$	6
6	Front Staircase: Rounds 0–3	6
7	One-Page STAIRCASE Visibility Summary	8
7.1	State-word visibility by round	8
7.2	Visibility graph	9
7.3	Extended backward visibility including T_1, T_2, Maj	9
7.4	Overall structural visibility	10
8	Worked Example: <code>SHA256("hello")</code>	10
8.1	Recover the final internal state	10
9	General Backward Formulas	11
10	Backward Derivation for Rounds 63 to 57	11
11	Complete Backward Staircase Table	12

12 Why h_{63} Is Not Determined by the Digest Alone	14
13 Future Scope of STAIRCASE	14
14 Conclusion	15
15 References	15
A Python Verification Script	16
A.1 How to run	16
A.2 Script	16
Author Details	20

1 Plain-Language Summary

Imagine SHA-256 as 64 locked rooms in a long corridor.

Each room has eight windows:

$$a, b, c, d, e, f, g, h.$$

Normally, after hashing, you only see the final exit door: the 256-bit hash.

The **STAIRCASE** discovery shows that some windows are already open:

- At the very beginning, because SHA-256 starts with public constants, the first few rooms have many open windows. This is called the **front staircase**.
- At the very end, because the final hash reveals the final internal state, we can walk backward and open many windows in the last seven rooms. This is called the **backward staircase**.
- The middle rooms remain mostly dark, because the message and nonlinear mixing hide them.

STAIRCASE gives a clear map:

which windows are open, which are closed, and how much is visible.

This is not a claim that SHA-256 is broken. It is a structural map of what can be known without the message.

2 Scope and Correctness Statement

This document proves and illustrates the following precise statement:

For standard SHA-256, starting from the standard IV and producing a single-block digest, the public constants and final digest determine a fixed pattern of internal state words in rounds 0–3 and rounds 57–63. This pattern is called the **STAIRCASE**. One important value, h_{63} , remains undetermined by the digest alone.

All percentages in this document are **structural visibility percentages**. They describe how many fixed state-word positions are known under the model. They do **not** mean that SHA-256 is broken by that percentage.

For multi-block messages, the same algebra applies to the final block, but the input chaining value of that block replaces the IV. If that chaining value is unknown, the first backward step cannot be performed from the digest alone.

All arithmetic is modulo 2^{32} . All hex values are 32-bit big-endian words.

3 Known Prior Work and Original Contribution

Professional cryptographic research must clearly separate **known foundations** from **new contribution**.

The purpose of citing previous work is not to reduce the author’s contribution. It is to show exactly where the public foundations end and where **STAIRCASE** begins.

Already known in public literature	Original contribution of STAIRCASE by Neerugattu Venkatesh
SHA-256 round function, IV, and round constants are defined by FIPS 180-4.	STAIRCASE names and presents a unified round-visibility structure combining front and backward visibility in one framework.
The first internal state is the IV.	The front staircase for rounds 0–3 is explicitly visualized, including the fixed value $T_{2,0} = 08909AE5$.
Final digest addition allows recovery of the final internal state for a single block.	The backward staircase for rounds 57–63 is presented as a step-by-step worked derivation with exact formulas.
Meet-in-the-middle and biclique ideas exist in prior cryptanalysis.	STAIRCASE provides a clear structural map that can be used as a future backward anchor, teaching tool, and constraint-filter framework.
Some backward substitution ideas are known informally in cryptanalysis.	STAIRCASE gives a beginner-friendly, professionally structured, percentage-based visibility exposition, to the best of the author’s knowledge not previously published in this exact form.

Table 1: Separation of known foundations and the author’s original contribution.

4 SHA-256 Round Function

For round $t = 0, \dots, 63$, SHA-256 uses eight 32-bit working variables:

$$(a_t, b_t, c_t, d_t, e_t, f_t, g_t, h_t).$$

The initial state for the first block is the SHA-256 IV:

$$(a_0, b_0, c_0, d_0, e_0, f_0, g_0, h_0) = (H_0, H_1, H_2, H_3, H_4, H_5, H_6, H_7).$$

The Boolean functions are:

$$\text{Ch}(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z),$$

$$\text{Maj}(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z).$$

The uppercase sigma functions are:

$$\Sigma_0(x) = \text{ROTR}^2(x) \oplus \text{ROTR}^{13}(x) \oplus \text{ROTR}^{22}(x),$$

$$\Sigma_1(x) = \text{ROTR}^6(x) \oplus \text{ROTR}^{11}(x) \oplus \text{ROTR}^{25}(x).$$

Here $\text{ROTR}^n(x)$ denotes a 32-bit right rotation by n bits.

For each round t , compute:

$$T_{1,t} = h_t + \Sigma_1(e_t) + \text{Ch}(e_t, f_t, g_t) + K_t + W_t \pmod{2^{32}},$$

$$T_{2,t} = \Sigma_0(a_t) + \text{Maj}(a_t, b_t, c_t) \pmod{2^{32}}.$$

Then update:

$$\begin{aligned}
a_{t+1} &= T_{1,t} + T_{2,t}, \\
b_{t+1} &= a_t, \\
c_{t+1} &= b_t, \\
d_{t+1} &= c_t, \\
e_{t+1} &= d_t + T_{1,t}, \\
f_{t+1} &= e_t, \\
g_{t+1} &= f_t, \\
h_{t+1} &= g_t.
\end{aligned}$$

After 64 rounds, for a single block, the digest words are:

$$D_i = H_i + s_{64,i} \pmod{2^{32}},$$

where

$$(s_{64,0}, \dots, s_{64,7}) = (a_{64}, b_{64}, c_{64}, d_{64}, e_{64}, f_{64}, g_{64}, h_{64}).$$

Therefore:

$$s_{64,i} = D_i - H_i \pmod{2^{32}}.$$

5 Constants Fixed for Every Standard SHA-256 First Block

5.1 Initial hash values

The SHA-256 IV is fixed:

Word	Hexadecimal value
H_0	6A09E667
H_1	BB67AE85
H_2	3C6EF372
H_3	A54FF53A
H_4	510E527F
H_5	9B05688C
H_6	1F83D9AB
H_7	5BE0CD19

Table 2: SHA-256 initial hash values.

5.2 Round constants

The 64 round constants K_t are also fixed. They are listed below in row-major order: the first row contains $K_0, \dots, K_7$, the second row contains $K_8, \dots, K_{15}$, and so on.

428A2F98	71374491	B5C0FBCF	E9B5DBA5	3956C25B	59F111F1	923F82A4	AB1C5ED5
D807AA98	12835B01	243185BE	550C7DC3	72BE5D74	80DEB1FE	9BDC06A7	C19BF174
E49B69C1	EFBE4786	0FC19DC6	240CA1CC	2DE92C6F	4A7484AA	5CB0A9DC	76F988DA
983E5152	A831C66D	B00327C8	BF597FC7	C6E00BF3	D5A79147	06CA6351	14292967
27B70A85	2E1B2138	4D2C6DFC	53380D13	650A7354	766A0ABB	81C2C92E	92722C85
A2BFE8A1	A81A664B	C24B8B70	C76C51A3	D192E819	D6990624	F40E3585	106AA070
19A4C116	1E376C08	2748774C	34B0BCB5	391C0CB3	4ED8AA4A	5B9CCA4F	682E6FF3
748F82EE	78A5636F	84C87814	8CC70208	90BEFFFA	A4506CEB	BEF9A3F7	C67178F2

Table 3: SHA-256 round constants $K_0, \dots, K_{63}$.

5.3 A first-round constant: $T_{2,0}$

Because the first-round state is the IV, $T_{2,0}$ is fixed for every standard SHA-256 first block.

We have:

$$a_0 = 6A09E667, \quad b_0 = BB67AE85, \quad c_0 = 3C6EF372.$$

Compute:

$$\Sigma_0(a_0) = \Sigma_0(6A09E667) = CE20B47E.$$

Also:

$$\text{Maj}(a_0, b_0, c_0) = \text{Maj}(6A09E667, BB67AE85, 3C6EF372) = 3A6FE667.$$

Therefore:

$$T_{2,0} = CE20B47E + 3A6FE667 = 08909AE5 \pmod{2^{32}}.$$

Thus:

$T_{2,0} = 08909AE5$

6 Front Staircase: Rounds 0–3

The following table shows the fixed IV-derived words in the first four rounds. A question mark means the value is not fixed by the public constants alone.

Round	a	b	c	d	e	f	g	h	T_1	T_2
0	6A09E667	BB67AE85	3C6EF372	A54FF53A	510E527F	9B05688C	1F83D9AB	5BE0CD19	?	08909AE5
1	?	6A09E667	BB67AE85	3C6EF372	?	510E527F	9B05688C	1F83D9AB	?	?
2	?	?	6A09E667	BB67AE85	?	?	510E527F	9B05688C	?	?
3	?	?	?	6A09E667	?	?	?	510E527F	?	?

Table 4: Front STAIRCASE for rounds 0–3.

The fixed values are:

$$\begin{aligned} b_1 &= 6A09E667, & c_1 &= BB67AE85, & d_1 &= 3C6EF372, \\ f_1 &= 510E527F, & g_1 &= 9B05688C, & h_1 &= 1F83D9AB, \\ c_2 &= 6A09E667, & d_2 &= BB67AE85, & g_2 &= 510E527F, \\ h_2 &= 9B05688C, & d_3 &= 6A09E667, & h_3 &= 510E527F. \end{aligned}$$

7 One-Page STAIRCASE Visibility Summary

This section gives a visual summary of how much internal state visibility is obtained from standard SHA-256 constants and the final digest.

The percentage is defined as:

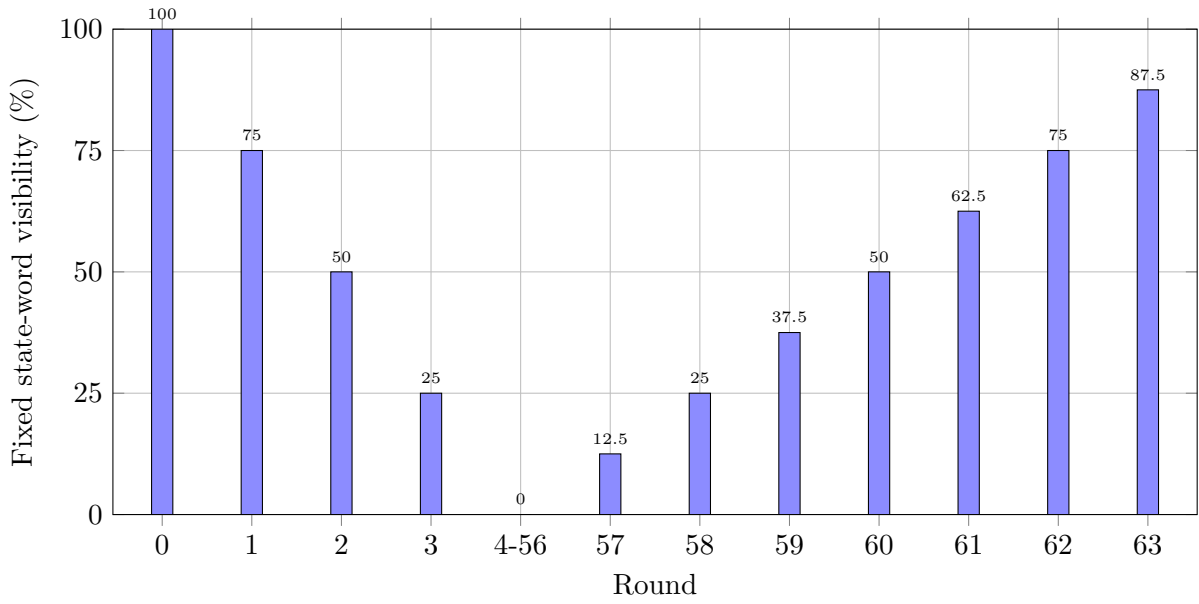
$$\text{visibility percentage} = \frac{\text{number of fixed state words}}{8} \times 100.$$

7.1 State-word visibility by round

Round	Fixed state words	Visibility	Reason
0	8/8	100%	Full IV known
1	6/8	75%	IV diagonal
2	4/8	50%	IV diagonal
3	2/8	25%	IV diagonal
4–56	0/8	0%	Message mixing hides state
57	1/8	12.5%	Backward staircase
58	2/8	25%	Backward staircase
59	3/8	37.5%	Backward staircase
60	4/8	50%	Backward staircase
61	5/8	62.5%	Backward staircase
62	6/8	75%	Backward staircase
63	7/8	87.5%	Backward staircase

Table 5: Fixed state-word visibility per round.

7.2 Visibility graph



7.3 Extended backward visibility including T_1, T_2, Maj

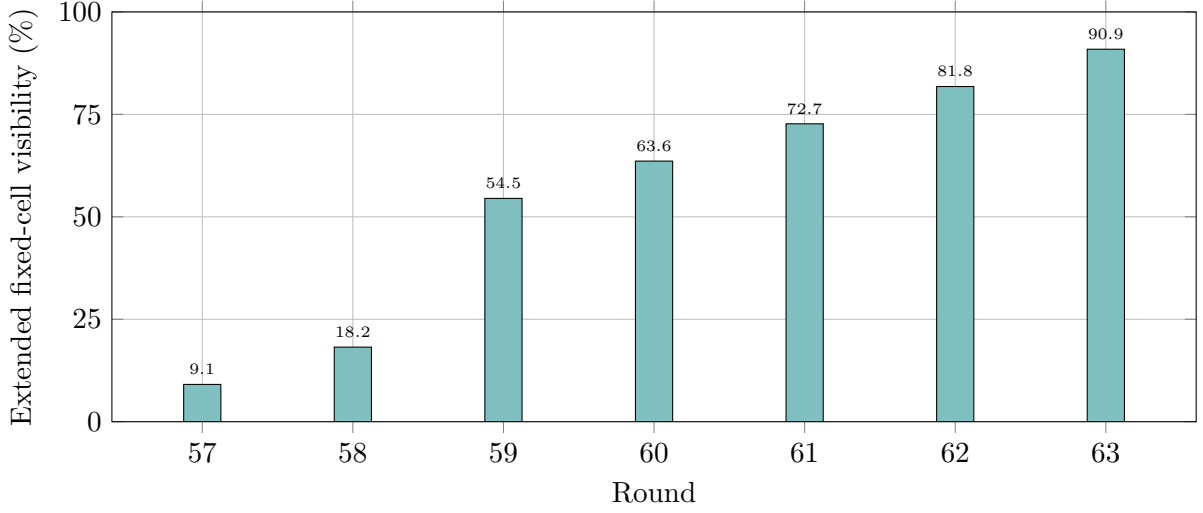
For the backward staircase, if we count the eleven cells

$$a, b, c, d, e, f, g, h, T_1, T_2, \text{Maj},$$

then the fixed-cell percentages for rounds 57–63 are:

Round	Fixed cells among 11	Visibility
57	1/11	9.1%
58	2/11	18.2%
59	6/11	54.5%
60	7/11	63.6%
61	8/11	72.7%
62	9/11	81.8%
63	10/11	90.9%

Table 6: Extended backward STAIRCASE visibility.



7.4 Overall structural visibility

Region	Fixed cells	Visibility
Front anchor, rounds 0–3, state words only	20/32	62.5%
Backward anchor, rounds 57–63, state words only	28/56	50.0%
Backward anchor, rounds 57–63, including T_1, T_2, Maj	43/77	55.8%
All 64 rounds, state words only	48/512	9.4%

Table 7: Overall structural visibility metrics.

Important:

These percentages describe structural visibility, not attack success.

8 Worked Example: SHA256("hello")

The known-answer test vector is:

$\text{SHA256}(\text{"hello"}) = 2\text{cf}24\text{db}a5\text{fb}0a30\text{e}26\text{e}83\text{b}2\text{ac}5\text{b}9\text{e}29\text{e}1\text{b}16\text{1e}5\text{c}1\text{fa}7425\text{e}73043362938\text{b}9824$.

8.1 Recover the final internal state

For a single block:

$$s_{64,i} = D_i - H_i \pmod{2^{32}}.$$

i	Digest word D_i	IV word H_i	Final state $s_{64,i} = D_i - H_i$
0	2CF24DBA	6A09E667	C2E86753
1	5FB0A30E	BB67AE85	A448F489
2	26E83B2A	3C6EF372	EA7947B8
3	C5B9E29E	A54FF53A	2069ED64
4	1B161E5C	510E527F	CA07CBDD
5	1FA7425E	9B05688C	84A1D9D2
6	73043362	1F83D9AB	538059B7
7	938B9824	5BE0CD19	37AACB0B

Table 8: Final SHA-256 state for "hello".

9 General Backward Formulas

From the SHA-256 update rule, if state $t + 1$ is known, then:

$$\boxed{\begin{array}{l} a_t = b_{t+1}, \\ b_t = c_{t+1}, \\ c_t = d_{t+1}, \\ e_t = f_{t+1}, \\ f_t = g_{t+1}, \\ g_t = h_{t+1}. \end{array}}$$

The word d_t is not directly copied, but it can be recovered if $T_{1,t}$ is known.

The key observation is that $T_{2,t}$ depends only on a_t, b_t, c_t :

$$T_{2,t} = \Sigma_0(a_t) + \text{Maj}(a_t, b_t, c_t).$$

Therefore, once a_t, b_t, c_t are known, $T_{2,t}$ is known. Then:

$$T_{1,t} = a_{t+1} - T_{2,t}.$$

Finally:

$$d_t = e_{t+1} - T_{1,t}.$$

The value h_t is not recoverable from this process alone, because h_t appears only inside $T_{1,t}$:

$$T_{1,t} = h_t + \Sigma_1(e_t) + \text{Ch}(e_t, f_t, g_t) + K_t + W_t.$$

If W_t is unknown, then h_t remains an algebraic degree of freedom. If h_t is known, then W_t can be solved, and vice versa.

10 Backward Derivation for Rounds 63 to 57

Let:

$$X = e_{60} = f_{61} = g_{62} = h_{63}.$$

The derived values are:

$$\begin{aligned} a_{63} &= \text{A448F489}, & b_{63} &= \text{EA7947B8}, & c_{63} &= \text{2069ED64}, \\ d_{63} &= \text{96168426}, & e_{63} &= \text{84A1D9D2}, & f_{63} &= \text{538059B7}, \\ g_{63} &= \text{37AACB0B}, & h_{63} &= X. \end{aligned}$$

$$T_{1,63} = \text{33F147B7}, \quad T_{2,63} = \text{8EF71F9C}, \quad \text{Maj}_{63} = \text{A069E5A8}.$$

$$\begin{aligned} a_{62} &= \text{EA7947B8}, & b_{62} &= \text{2069ED64}, & c_{62} &= \text{96168426}, \\ d_{62} &= \text{651A8BFA}, & e_{62} &= \text{538059B7}, & f_{62} &= \text{37AACB0B}. \end{aligned}$$

$$T_{1,62} = \text{1F874DD8}, \quad T_{2,62} = \text{84C1A6B1}, \quad \text{Maj}_{62} = \text{A279C524}.$$

$$\begin{aligned} a_{61} &= \text{2069ED64}, & b_{61} &= \text{96168426}, & c_{61} &= \text{651A8BFA}, \\ d_{61} &= \text{51B087FC}, & e_{61} &= \text{37AACB0B}. \end{aligned}$$

$$T_{1,61} = \text{01CFD1BB}, \quad T_{2,61} = \text{E8A975FD}, \quad \text{Maj}_{61} = \text{241A8D66}.$$

$$a_{60} = 96168426, \quad b_{60} = 651A8BFA, \quad c_{60} = 51B087FC, \\ d_{60} = 4AF4F18A, \quad e_{60} = X.$$

$$T_{1,60} = ECB5D981, \quad T_{2,60} = 33B413E3, \quad \text{Maj}_{60} = 551287FE.$$

$$a_{59} = 651A8BFA, \quad b_{59} = 51B087FC, \quad c_{59} = 4AF4F18A, \\ d_{59} = X - A7AB9C6E.$$

$$T_{1,59} = A7AB9C6E, \quad T_{2,59} = EE6AE7B8, \quad \text{Maj}_{59} = 41B083FA.$$

$$a_{58} = 51B087FC, \quad b_{58} = 4AF4F18A, \quad c_{58} = X - A7AB9C6E.$$

$$a_{57} = 4AF4F18A.$$

11 Complete Backward Staircase Table

The table below shows the fixed values derived from the digest. A question mark denotes a value that is not fixed by the digest alone.

Round	a	b	c	d	e	f	g	h	T_1	T_2	Maj
63	A448F489	EA7947B8	2069ED64	96168426	84A1D9D2	538059B7	37AACB0B	?	33F147B7	8EF71F9C	A069E5A8
62	EA7947B8	2069ED64	96168426	651A8BFA	538059B7	37AACB0B	?	?	1F874DD8	84C1A6B1	A279C524
61	2069ED64	96168426	651A8BFA	51B087FC	37AACB0B	?	?	?	01CFD1BB	E8A975FD	241A8D66
60	96168426	651A8BFA	51B087FC	4AF4F18A	?	?	?	?	ECB5D981	33B413E3	551287FE
59	651A8BFA	51B087FC	4AF4F18A	?	?	?	?	?	A7AB9C6E	EE6AE7B8	41B083FA
58	51B087FC	4AF4F18A	?	?	?	?	?	?	?	?	?
57	4AF4F18A	?	?	?	?	?	?	?	?	?	?

Table 9: Backward STAIRCASE for SHA256("hello"), rounds 57–63.

The fixed-cell pattern is:

Round	a	b	c	d	e	f	g	h	T_1	T_2	Maj
57	✓	?	?	?	?	?	?	?	?	?	?
58	✓	✓	?	?	?	?	?	?	?	?	?
59	✓	✓	✓	?	?	?	?	?	✓	✓	✓
60	✓	✓	✓	✓	?	?	?	?	✓	✓	✓
61	✓	✓	✓	✓	✓	?	?	?	✓	✓	✓
62	✓	✓	✓	✓	✓	✓	?	?	✓	✓	✓
63	✓	✓	✓	✓	✓	✓	✓	?	✓	✓	✓

12 Why h_{63} Is Not Determined by the Digest Alone

The round-63 equation for $T_{1,63}$ is:

$$T_{1,63} = h_{63} + \Sigma_1(e_{63}) + \text{Ch}(e_{63}, f_{63}, g_{63}) + K_{63} + W_{63}.$$

From the backward staircase we know:

$$T_{1,63}, \quad e_{63}, \quad f_{63}, \quad g_{63}.$$

The round constant K_{63} is public. But the digest alone does not give both h_{63} and W_{63} .

Rearranging:

$$h_{63} + W_{63} = T_{1,63} - \Sigma_1(e_{63}) - \text{Ch}(e_{63}, f_{63}, g_{63}) - K_{63}.$$

Thus the digest determines a relation between h_{63} and W_{63} , but not each value separately.

If W_{63} is known, then h_{63} can be solved. If h_{63} is known, then W_{63} can be solved. Without one of them, one algebraic degree of freedom remains.

13 Future Scope of STAIRCASE

The STAIRCASE framework can be used as a foundation for future work.

Possible directions include:

1. **Educational use:** STAIRCASE makes SHA-256 internal structure visible and understandable.
2. **Constraint filtering:** The derived T_1, T_2, Maj values can reject invalid internal-state guesses.
3. **Backward anchor for reduced-round analysis:** The staircase gives a clean backward starting point for rounds 57–63.
4. **Combination with meet-in-the-middle frameworks:** Future research may study whether STAIRCASE can be combined with neutral words, bicliques, or partial-fixing techniques.
5. **Symbolic solving:** The unknown $X = h_{63}$ can be treated as a symbolic variable in larger constraint systems.
6. **Message-schedule bridges:** Future work may investigate how the staircase constraints interact with the SHA-256 message schedule across the middle rounds.

Important:

STAIRCASE is a structural contribution, not a full SHA-256 break.

14 Conclusion

STAIRCASE, introduced by **Neerugattu Venkatesh**, provides a clear structural decomposition of SHA-256 round visibility.

It shows:

- the front staircase from the IV in rounds 0–3;
- the backward staircase from the final digest in rounds 57–63;
- the fixed first-round value $T_{2,0} = 08909AE5$;
- the exact percentage of fixed state-word positions;
- the main unknown value $X = h_{63}$;
- a professional foundation for future cryptographic research.

The contribution is original in its explicit naming, visualization, percentage-based exposition, and beginner-friendly proof structure, while respecting and citing the known cryptographic foundations.

15 References

1. NIST, *FIPS PUB 180-4: Secure Hash Standard (SHS)*, 2015.
2. Y. Sasaki, X. Wang, K. Aoki, *Preimage Attacks on 41-Step SHA-256*, ASIACRYPT 2009.
3. J. Li, T. Isobe, K. Shibutani, *Converting Meet-in-the-Middle Preimage Attack into Pseudo Collision Attack: Application to SHA-2*, FSE 2012.
4. D. Khovratovich, C. Rechberger, A. Savelieva, *Bicliques for Preimages: Attacks on Skein-512 and the SHA-2 Family*, FSE 2012.

A Python Verification Script

The following Python script verifies the main numeric claims in this document. It checks:

- the SHA-256 digest of "hello";
- recovery of the final internal state;
- the fixed value $T_{2,0} = 08909AE5$;
- the front staircase diagonal constants;
- the backward staircase values for rounds 63 down to 57.

This updated version pauses before closing, so it works comfortably on Windows, macOS, and Linux.

A.1 How to run

Save the script as:

```
1 STAIRCASE_SHA256_verify.py
```

Then run:

```
1 python3 STAIRCASE_SHA256_verify.py
```

On Windows, you can also run:

```
1 py STAIRCASE_SHA256_verify.py
```

Or simply double-click the file if Python is installed.

A successful run ends with:

```
1 ALL CHECKS PASSED
2 Press Enter to exit...
```

A.2 Script

Listing 1: STAIRCASE SHA-256 verification script — cross-platform version

```
1 #!/usr/bin/env python3
2 # -*- coding: utf-8 -*-
3
4 # STAIRCASE_SHA256_verify.py
5 # Author: Neerugattu Venkatesh
6 # Email: venkateshneerugattu7@gmail.com
7 # GitHub: https://github.com/venkateshneerugattu7-commits
8
9 import sys
10 import traceback
11 from hashlib import sha256
12
13 MASK = 0xFFFFFFFF
14
15
16 def rotr(x, n):
17     return ((x >> n) | (x << (32 - n))) & MASK
18
19
20 def add_mod(*xs):
21     s = 0
22     for x in xs:
23         s += x
24     return s & MASK
```

```

25
26
27 def sub_mod(x, y):
28     return (x - y) & MASK
29
30
31 def Sigma0(x):
32     return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)
33
34
35 def Sigma1(x):
36     return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)
37
38
39 def Ch(x, y, z):
40     return ((x & y) ^ ((~x) & z)) & MASK
41
42
43 def Maj(x, y, z):
44     return ((x & y) ^ (x & z) ^ (y & z)) & MASK
45
46
47 def hx(x):
48     return f"0x{x:08X}"
49
50
51 def check(name, got, exp):
52     ok = got == exp
53     print(f"{name:28s} got={hx(got)} exp={hx(exp)} {'OK' if ok else 'FAIL'}")
54     assert ok, name
55
56
57 def main():
58     IV = [
59         0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
60         0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
61     ]
62
63     print("STAIRCASE SHA-256 verification")
64     print("Author: Neerugattu Venkatesh")
65     print()
66
67     digest = sha256(b"hello").hexdigest()
68     expected_digest = "2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824"
69     print("Digest:", digest)
70     assert digest == expected_digest, "Digest mismatch"
71
72     D = [int(digest[i:i + 8], 16) for i in range(0, 64, 8)]
73     S64 = [(D[i] - IV[i]) & MASK for i in range(8)]
74
75     expected_S64 = [
76         0xC2E86753, 0xA448F489, 0xEA7947B8, 0x2069ED64,
77         0xCA07CBDD, 0x84A1D9D2, 0x538059B7, 0x37AACB0B,
78     ]
79
80     for i, (got, exp) in enumerate(zip(S64, expected_S64)):
81         check(f"S64[{i}]", got, exp)
82
83     # Front staircase diagonal constants
84     check("b1", IV[0], 0x6A09E667)
85     check("c1", IV[1], 0xBB67AE85)
86     check("d1", IV[2], 0x3C6EF372)
87     check("f1", IV[4], 0x510E527F)
88     check("g1", IV[5], 0x9B05688C)
89     check("h1", IV[6], 0x1F83D9AB)
90
91     check("c2", IV[0], 0x6A09E667)
92     check("d2", IV[1], 0xBB67AE85)
93     check("g2", IV[4], 0x510E527F)
94     check("h2", IV[5], 0x9B05688C)
95
96     check("d3", IV[0], 0x6A09E667)
97     check("h3", IV[4], 0x510E527F)

```

```

98
99 # First-round constant T2_0
100 T2_0 = add_mod(Sigma0(IV[0]), Maj(IV[0], IV[1], IV[2]))
101 check("T2_0", T2_0, 0x08909AE5)
102
103 a64, b64, c64, d64, e64, f64, g64, h64 = S64
104
105 # Round 63
106 a63 = b64
107 b63 = c64
108 c63 = d64
109 e63 = f64
110 f63 = g64
111 g63 = h64
112
113 T2_63 = add_mod(Sigma0(a63), Maj(a63, b63, c63))
114 T1_63 = sub_mod(a64, T2_63)
115 d63 = sub_mod(e64, T1_63)
116 Maj63 = sub_mod(T2_63, Sigma0(a63))
117
118 check("a63", a63, 0xA448F489)
119 check("b63", b63, 0xEA7947B8)
120 check("c63", c63, 0x2069ED64)
121 check("d63", d63, 0x96168426)
122 check("e63", e63, 0x84A1D9D2)
123 check("f63", f63, 0x538059B7)
124 check("g63", g63, 0x37AACB0B)
125 check("T1_63", T1_63, 0x33F147B7)
126 check("T2_63", T2_63, 0x8EF71F9C)
127 check("Maj63", Maj63, 0xA069E5A8)
128
129 # Round 62
130 a62 = b63
131 b62 = c63
132 c62 = d63
133 e62 = f63
134 f62 = g63
135
136 T2_62 = add_mod(Sigma0(a62), Maj(a62, b62, c62))
137 T1_62 = sub_mod(a63, T2_62)
138 d62 = sub_mod(e63, T1_62)
139 Maj62 = sub_mod(T2_62, Sigma0(a62))
140
141 check("a62", a62, 0xEA7947B8)
142 check("b62", b62, 0x2069ED64)
143 check("c62", c62, 0x96168426)
144 check("d62", d62, 0x651A8BFA)
145 check("e62", e62, 0x538059B7)
146 check("f62", f62, 0x37AACB0B)
147 check("T1_62", T1_62, 0x1F874DD8)
148 check("T2_62", T2_62, 0x84C1A6B1)
149 check("Maj62", Maj62, 0xA279C524)
150
151 # Round 61
152 a61 = b62
153 b61 = c62
154 c61 = d62
155 e61 = f62
156
157 T2_61 = add_mod(Sigma0(a61), Maj(a61, b61, c61))
158 T1_61 = sub_mod(a62, T2_61)
159 d61 = sub_mod(e62, T1_61)
160 Maj61 = sub_mod(T2_61, Sigma0(a61))
161
162 check("a61", a61, 0x2069ED64)
163 check("b61", b61, 0x96168426)
164 check("c61", c61, 0x651A8BFA)
165 check("d61", d61, 0x51B087FC)
166 check("e61", e61, 0x37AACB0B)
167 check("T1_61", T1_61, 0x01CFD1BB)
168 check("T2_61", T2_61, 0xE8A975FD)
169 check("Maj61", Maj61, 0x241A8D66)
170

```

```

171     # Round 60
172     a60 = b61
173     b60 = c61
174     c60 = d61
175
176     T2_60 = add_mod(Sigma0(a60), Maj(a60, b60, c60))
177     T1_60 = sub_mod(a61, T2_60)
178     d60 = sub_mod(e61, T1_60)
179     Maj60 = sub_mod(T2_60, Sigma0(a60))
180
181     check("a60", a60, 0x96168426)
182     check("b60", b60, 0x651A8BFA)
183     check("c60", c60, 0x51B087FC)
184     check("d60", d60, 0x4AF4F18A)
185     check("T1_60", T1_60, 0xECB5D981)
186     check("T2_60", T2_60, 0x33B413E3)
187     check("Maj60", Maj60, 0x551287FE)
188
189     # Round 59
190     a59 = b60
191     b59 = c60
192     c59 = d60
193
194     T2_59 = add_mod(Sigma0(a59), Maj(a59, b59, c59))
195     T1_59 = sub_mod(a60, T2_59)
196     Maj59 = sub_mod(T2_59, Sigma0(a59))
197
198     check("a59", a59, 0x651A8BFA)
199     check("b59", b59, 0x51B087FC)
200     check("c59", c59, 0x4AF4F18A)
201     check("T1_59", T1_59, 0xA7AB9C6E)
202     check("T2_59", T2_59, 0xEE6AE7B8)
203     check("Maj59", Maj59, 0x41B083FA)
204
205     # Round 58
206     a58 = b59
207     b58 = c59
208
209     check("a58", a58, 0x51B087FC)
210     check("b58", b58, 0x4AF4F18A)
211
212     # Round 57
213     a57 = d60
214     check("a57", a57, 0x4AF4F18A)
215
216     print()
217     print("ALL CHECKS PASSED")
218
219
220 if __name__ == "__main__":
221     exit_code = 0
222
223     try:
224         main()
225     except KeyboardInterrupt:
226         print()
227         print("Interrupted by user.")
228         exit_code = 130
229     except Exception:
230         print()
231         traceback.print_exc()
232         print()
233         print("VERIFICATION FAILED")
234         exit_code = 1
235     finally:
236         try:
237             print()
238             input("Press Enter to exit...")
239         except (EOFError, KeyboardInterrupt):
240             pass
241
242     sys.exit(exit_code)

```

Author Details

Neerugattu Venkatesh

Freelancer / Independent Cryptography Researcher

Research Contribution: **STAIRCASE**

Email: venkateshneerugattu7@gmail.com

GitHub: <https://github.com/venkateshneerugattu7-commits>

This document presents the author's original structural contribution, **STAIRCASE**, to the cryptographic community. The work respects known SHA-256 literature while clearly identifying the new exposition, visualization, and verification framework introduced here.